le cnam

Conservatoire national des arts et métiers

École doctorale Sciences des Métiers de l'Ingénieur

Laboratoire de Mécanique des Structures et des Systèmes Couplés

Thèse de doctorat

Présentée par : **Mathieu Aucejo**

Soutenue le : **15 juin 2024**

Discipline : **Sciences de l'ingénieur**

Spécialité : **Mécanique**

Guide d'utilisation du gabarit Typst pour les thèses du Cnam

Thèse dirigée par :

Henri Grégoire, Abbé constitutionnelle, Cnam, Paris

Henri Tresca, Professeur titulaire de la Chaire de Mécanique, Cnam, Paris

Et co-dirigée par :

Pierre-Simon Laplace, Professeur de tout, Institut de France, Paris

Joseph Fourier, Membre de l'Académie des Sciences, Académie des sciences, Paris

Jury

Jean-Antoine Chaptal	Professeur des Unversités, Chaire de Chimie Appliquée, Conservatoire national des arts et métiers, Paris, France	Président du jury
Donald Ervin Knuth	Professeur émérite, Département d'Informatique, Université de Stanford, Palo, Alto, Californie, États-Unis	Rapporteur
Ada Lovelace	Maître de conférences HDR, Département de Mathématiques, Université de Londres, Royaume-Uni	Rapportrice
Jacques de Vaucanson	Inspecteur général des manufactures, Académie royale des sciences, Paris, France	Examinateur
Henri Grégoire	Abbé constitutionnelle, Conservatoire national des arts et métiers, Paris, France	Directeur de thèse
Henri Tresca	Professeur titulaire de la Chaire de Mécanique, Conservatoire national des arts et métiers, Paris, France	Co-directeur de thèse

Les logiciels et les cathédrales, c'est un peu la même chose, d'abord on les construit, ensuite on prie.
Sam Redwine

Avant-propos

Ce document présente le guide d'utilisation du template Typst pour les thèses du Conservatoire national des arts et métiers (Cnam). Il a été conçu pour aider les doctorants et les étudiants de Master à rédiger leur thèse en utilisant le logiciel Typst, qui offre une alternative moderne et efficace à LaTeX.

Ce guide exploite l'ensemble des fonctionnalités offertes par le template, et fournit des exemples concrets pour illustrer son utilisation. Il est destiné à aider les doctorants et les étudiants de Master à rédiger leur thèse de manière efficace et professionnelle, en respectant les normes et les exigences du Cnam. Typst pour leur travail remarquable sur ce logiciel. Un remerciement particulier est également adressé à la communauté Typst, sans laquelle ce template n'aurait pu voir le jour.

Je remercie également mon collègue Éric Bavu pour m'avoir motivé à créer ce template en publiant un template Quarto + LaTeX pour les thèses du Cnam. Le template Typst s'inspire de son travail. Merci Éric !

Enfin, je remercie les doctorants et les étudiants qui ont utilisé ou utiliseront ce template pour leurs retours et suggestions, qui permettront d'améliorer continuellement cet outil pour le bénéfice de tous.

Mathieu Aucejo
Juin 2026

Résumé

Ce travail présente et documente le template Typst `community-cnam-thesis`, conçu pour permettre aux doctorants du Conservatoire national des arts et métiers (Cnam) de rédiger leur thèse dans un environnement moderne, lisible et reproductible. Le template vise la production d'un PDF conforme à la maquette institutionnelle 2024-2025, avec une attention particulière à la qualité typographique, à la structuration du manuscrit et à la compatibilité avec les exigences d'archivage (notamment PDF/A). Le guide se compose de six chapitres, chacun dédié à une grande famille de fonctionnalités : structure, figures et tableaux, équations, boîtes informationnelles, ainsi que références et annotations. La plupart des contenus suivent le principe « code + rendu » : un extrait Typst présente la syntaxe, puis son résultat est affiché immédiatement dans le document. Cette logique permet de considérer non seulement comme un guide pratique d'utilisation, mais également comme une démonstration des possibilités offertes par le template.

Mots-clés : Typst, template, thèse, Cnam, PDF/A, publication scientifique reproductible, guide d'utilisation, documentation.

Abstract

This work presents and documents the Typst template `community-cnam-thesis`, designed to enable doctoral candidates at the Conservatoire national des arts et métiers (Cnam) to write their thesis in a modern, readable, and reproducible environment. The template aims to produce a PDF compliant with the 2024-2025 institutional format, with particular attention to typographic quality, manuscript structure, and compatibility with archiving requirements (notably PDF/A). The guide consists of six chapters, each devoted to a major feature category: structure, figures and tables, equations, informational boxes, as well as references and annotations. Most of the content follows the "code + output" principle: a Typst excerpt presents the syntax, and its result is displayed immediately in the document. This approach allows it to be considered not only as a practical user guide, but also as a demonstration of the possibilities offered by the template.

Keywords: Typst, template, thesis, Cnam, PDF/A, reproducible scientific publishing, user guide, documentation.

Table des matières

Avant-propos i

Résumé iii

Abstract v

Guide d'utilisation 11

Introduction . . . 13
Contexte et objectifs . . . 13
Structure du guide . . . 13
Présentation de Typst . . . 14
Installation et utilisation de Typst-CLI . . . 14
Installation et utilisation des utilitaires . . . 16
Éditeurs de texte et IDE . . . 16
Gestionnaire de paquets . . . 18

1. Usage général . . . 21
1.1. Informations générales . . . 22
1.1.1. Polices de caractères . . . 22
1.1.2. Couleurs du thème . . . 22
1.2. Initialisation du gabarit . . . 22
1.3. Arborescence des fichiers . . . 26
1.4. Structure du fichier principal . . . 26

2. Mise en forme du document . . . 29
2.1. Hiérarchisation des titres . . . 30
2.1.1. Chapitres numérotés et non numérotés . . . 30
2.1.2. Sections numérotées et non numérotées . . . 30
2.2. Références croisées . . . 31
2.3. Mise en forme du texte . . . 31
2.3.1. Emphase et code . . . 31
2.3.2. Liens hypertextes . . . 31
2.3.3. Listes . . . 31
2.3.4. Citations . . . 32
2.3.5. Notes de bas de pages . . . 32

2.3.6. Sauts de page . . . 32
2.4. Parties et environnements de mise en page . . . 33
2.5. Quatrième de couverture . . . 34

3. Figures et tableaux . . . 35
3.1. Figures . . . 36
3.2. Tableaux . . . 37
3.3. Titres longs et courts pour les figures et tableau . . . 39
3.4. Création de courbes directement en Typst . . . 39

4. Équations et algorithmes . . . 43
4.1. Équations . . . 44
4.1.1. Équations en ligne . . . 44
4.1.2. Équations numérotées et non numérotées . . . 44
4.1.3. Environnements avancés . . . 45
4.2. Algorithmes . . . 46

5. Boîtes informationnelles . . . 49
5.1. Boîtes disponibles . . . 50
5.2. Cas particuliers des boîtes de code . . . 51

6. Références . . . 53
6.1. Références bibliographiques . . . 54
6.1.1. Format de fichiers . . . 54
6.1.2. Syntaxe de base . . . 54
6.1.3. Références multiples . . . 54
6.2. Glossaire, acronymes et nomenclatures . . . 55

7. Commentaires de relecture . . . 57
7.1. Commentaires collaboratifs . . . 58
7.2. Types de commentaires . . . 58
7.3. Syntaxe des annotations . . . 59
7.4. Table des annotations . . . 61

Conclusion . . . 63
Récapitulatif des fonctionnalités . . . 63

Bibliographie . . . 65

Annexes **67**

A. Validation PDF/A-1b . . . 69
A.1. Dépôt sur theses.fr . . . 70

A.2. Validation CINES 70
A.3. Compilation Typst en PDF/A-1b 70

Table des figures

Figure 3.1. Logo officiel du Cnam 36

Figure 3.2. (a) Logo du Cnam and (b) Estampille de la Victoire 37

Figure 3.3. Représentation d'un rectangle 39

Figure 3.4. Réponse libre d'un système mécanique à un degré de liberté 41

Figure A.1. Sélection du format PDF/A-1b dans l'application web de Typst 70

Figure A.2. Sélection du format PDF/A-1b dans l'extension Tinymist de Visual Studio Code 71

Table des tableaux

Tableau 3.1. Tableau des périmètres de quelques formes géométriques 38

Commentaires de relecture

1 **Abbé Grégoire :** Ceci est un commentaire bleu. 60

2 **Henri Tresca :** Je trouve cela génial ! 60

3 **Abbé Grégoire :** Je ne suis pas sûr de comprendre cette remarque Henri ? 60

4 **Abbé Grégoire :** Comme je n'ai pas d'idées pour faire un texte amusant et long, je place un classique `#lorem()`. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat. 60

5 **Mathieu Aucejo :** Je me demande si cela est vraiment nécessaire ? 61

Résumé des commentaires de relecture

Note	2
Comment	1
Question	2
Todo	0
Total	5

Guide d’utilisation

Introduction

Contexte et objectifs

Les doctorants du Conservatoire national des arts et métiers (Cnam) doivent rédiger leur thèse en respectant certaines normes et exigences. Le format de la thèse, la mise en page, les polices de caractères, les couleurs et d'autres éléments doivent être conformes aux directives du Cnam. Cependant, tous ne souhaitent pas utiliser LaTeX, qui est souvent considéré comme complexe à utiliser et difficile à apprendre. Depuis 2023, Typst est est apparu comme une alternative moderne et efficace à LaTeX. Typst est un langage de balise qui permet de créer des documents de haute qualité avec une syntaxe simple et intuitive. Il est particulièrement adapté à la rédaction de thèses, car il permet de gérer facilement les références, les figures, les tableaux et les équations.

Les objectifs de ce guide sont de présenter le gabarit Typst pour les thèses du Cnam, ainsi que de fournir des instructions détaillées sur son utilisation. Ce guide n'a pas pour vocation de fournir une introduction complète à Typst, mais plutôt de se concentrer sur les aspects spécifiques à la rédaction à ce gabarit.

Pour plus d'information sur Typst, vous pouvez consulter le site officiel de Typst. La documentation officielle de Typst est très complète et fournit des exemples et des explications détaillées sur les différentes fonctionnalités du langage. Vous pouvez aussi consulter le livre Typst Handbook: From Zero to Hero de Stefano Coelati Rama, qui est un guide complet pour apprendre Typst.

Structure du guide

Ce guide est organisé en sept chapitres thématiques :

1. **Usage général** – Instruction générales pour l'utilisation du gabarit `community-cnam-thesis`.
2. **Mise en forme** – Hiérarchie des titres, numérotation, références croisées, listes, etc.
3. **Figures et tableaux** – Insertion d'images, figures, tableaux, légendes, mise en forme, figures multi-colonnes, etc.
4. **Équations et algorithmes** – Syntaxe des équations équations et insertion d'algorithmes.
5. **Boîtes informationnelles** – Types de boîtes, mise en forme, couleurs, icônes, etc.
6. **Références et glossaire** – Bibliographie, formats de fichier, création d'un glossaire, etc.
7. **Commentaires de relecture** – Notes de marge, commentaires, etc.

Présentation de Typst

Typst est un nouveau langage de balise open source é crit en Rust et développé à partir de 2019 par deux étudiants allemands, Laurenz Mädje et Martin Haug. La version 0.1.0 a été publiée sur GitHub le 04 avril 2023[1].

Typst se présente comme un successeur de LaTeX plus moderne, rapide et simple d'utilisation. Parmi ses avantages, on peut citer :

- la compilation incrémentale ;
- des messages d'erreur clair et compréhensible ;
- un langage de programmation Turing-complet ;
- une système de style cohérent permettant de configurer aisément tous les aspects de son document (police, pagination, marges, …) ;
- une communauté active et sympathique (serveur Discord pour le support, annonce de nouveaux paquets) ;
- un système de paquets simple d'utilisation (pour rechercher ou voir la liste des paquets, n'hésitez pas à visiter Typst Universe ;
- des extensions pour VS Code existent, comme `Tinymist`. Celles-ci offrent des fonctionnalités similaires à `LaTeX Workshop`.

Installation et utilisation de Typst-CLI

L'installation de Typst-CLI[2] est simple et rapide et est disponible pour Windows, macOS et Linux.

- Compilation à partir des sources : Si vous souhaitez compiler Typst à partir des sources, vous pouvez suivre les instructions disponibles sur le dépôt GitHub du projet. Vous aurez besoin d'installer Rust et Cargo. Une fois fait, vous pouvez utiliser la commande suivante dans votre terminal :

Code

```
cargo install --locked typst-cli
```

- Windows : Plusieurs méthodes sont disponibles pour installer Typst sur Windows. Vous pouvez télécharger le binaire sur le dépôt GitHub de projet, ou utiliser un gestionnaire de paquets comme Chocolatey, Scoop ou Winget. Pour installer Typst avec ses gestionnaires de paquets, vous pouvez utiliser les commandes suivantes dans votre terminal :

[1] Adresse du dépôt GitHub : https://github.com/typst/typst
[2] Utilitaire en ligne de commande

Code

```
choco install typst              # pour Chocolatey
scoop install main/typst         # pour Scoop
winget install --id Typst.Typst  # pour Winget
```

- Linux : Typst est disponible sur la plupart des gestionnaires de paquets associé aux grandes distributions. Vous pouvez l'installer avec les commandes suivantes dans votre terminal :

Code

```
sudo snap install typst                                    # Snapcraft (Ubuntu)
sudo dnf copr enable claaj/typst & sudo dnf install typst  # Fedora
sudo pacman -S typst                                       # Arch-based
```

- MacOS : Typst peut être installé sur macOS en utilisant les gestionnaire de paquets Homebrew ou MacPorts. Vous pouvez l'installer avec la commande suivante dans votre terminal :

Code

```
brew install typst
sudo port install typst
```

Une fois l'utilitaire Typst-CLI installé, vous pouvez compiler vos fichiers Typst en utilisant la commande suivante dans votre terminal :

Code

```
typst compile /chemin/vers/fichier.typ.                    # Sortie fichier.pdf
typst compile /chemin/vers/fichier.typ /chemin/vers/fichier_compile.pdf. # Sortie
fichier_compile.pdf
```

Typst-CLI offre également d'autres fonctionnalités, telles que la visualisation en temps réel de vos documents. Celle-ci surveille les modifications apportées à vos fichiers et recompile automatiquement le document. Pour l'utiliser, il faut ouvrir le PDF généré avec un lecteur PDF compatible avec la mise à jour automatique après avoir exécuté la commande suivante dans votre terminal :

Code

```
typst watch fichier.typ
```

Installation et utilisation des utilitaires

Plusieurs types d'utilitaires sont disponibles pour faciliter l'utilisation de Typst. Ces utilitaires concernent principalement les éditeurs de texte et les environnements de développement intégrés (IDE). Ces outils permettent d'améliorer l'expérience de l'utilisateur en offrant des fonctionnalités telles que la coloration syntaxique, l'autocomplétion, la compilation en temps réel et la visualisation du PDF généré.

Éditeurs de texte et IDE

La section précédente a présenté l'installation et l'utilisation de Typst-CLI qui permet de travailler en local avec Typst. Cette solution est idéale pour les utilisateurs avancés qui aiment travailler avec des éditeurs de texte et des terminaux. Cependant, pour les utilisateurs qui préfèrent utiliser un environnement de développement intégré (IDE) ou un éditeur de texte avec une interface graphique, il existe plusieurs options disponibles.

- **Éditeurs de texte**

  Typst peut être utilisé avec n'importe quel éditeur de texte, mais certains éditeurs offrent des fonctionnalités supplémentaires comme la coloration syntaxique. Libre à vous d'utiliser votre éditeur de texte préféré.

- **Visual Studio Code et ses dérivés**

  Visual Studio Code (VS Code) est un éditeur de code source développé par Microsoft. Il est gratuit, open source et disponible pour Windows, macOS et Linux. VS Code est largement utilisé par les développeurs principalement en raison de sa flexibilité, de sa personnalisation et de son large éventail d'extensions disponibles.

  Pour utiliser Typst avec VS Code, vous pouvez installer l'extension `Tinymist`. Cette extension offre des fonctionnalités similaires à celles de `LaTeX Workshop`, telles que la compilation en temps réel, la visualisation du PDF généré. Pour installer l'extension, vous pouvez suivre les étapes suivantes :

  - Ouvrir VS Code
  - Accéder au panneau des extensions
  - Rechercher `Tinymist`
  - Cliquer sur `Installer`

  > **Remarque**
  >
  > `Tinymist` embarque un compilateur Typst, ce qui signifie que vous n'avez pas besoin d'installer Typst-CLI séparément. Cependant, si vous souhaitez utiliser Typst-CLI pour compiler vos fichiers Typst, vous pouvez le faire en suivant les instructions de la section précédente.
  >
  > `Tinymist` est disponible sur tous les IDE dérivant de VS Code, tels que VS Codium, Cursor, WindSurf ou encore Positron.

- **IDE JetBrains**

  Typst peut également être utilisé avec les IDE de JetBrains, tels que IntelliJ IDEA, PyCharm, WebStorm, etc. Pour utiliser Typst avec ces IDE, vous pouvez installer l'extension `Typst Support` qui est basée sur `Tinymist`. Pour installer cette extension, vous pouvez suivre les étapes suivantes :

  - Ouvrir votre IDE JetBrains
  - Aller dans `Settings` > `Plugins`
  - Rechercher `Typst Support`
  - Cliquer sur `Install`

- **IDE cloud**

Malgré sa jeunesse, Typst dispose d'un IDE cloud officiel. Cet IDE dispose d'une offre gratuite généreuse pour une utilisation personnelle. Des alternatives à l'IDE cloud officiel existent, comme TypeTeX.

Gestionnaire de paquets

Contrairement à LaTeX, Typst ne fournit de distribution de paquets à l'instar de TeXLive ou MikTeX. Les paquets Typst sont centralisés sur un dépôt GitHub et peuvent être trouvés via le moteur de recherche Typst Universe[3].

Pour installer un paquet Typst, il vous suffit simplement de l'importer dans votre fichier Typst en utilisant l'une des syntaxes suivantes :

Code

```
#import "@preview/nom-du-paquet:version": *
#import "@preview/nom-du-paquet:version": nom-du-paquet
#import "@preview/nom-du-paquet:version": func1, func2
```

Prenons l'exemple du paquet `community-cnam-thesis` qui est utilisé dans ce guide. Pour l'importer, vous pouvez utiliser la syntaxe suivante :

Code

```
// Importe toutes les fonctions et variables du paquet
#import "@local/community-cnam-thesis:0.1.4": *

// Importe le nom du paquet pour l'utiliser dans votre code
// Par exemple, #ct.info-box[ ... ] crée une boîte d'information
#import "@preview/community-cnam-thesis:0.1.0": ct

// Importe uniquement les fonctions `info-box` et `warning-box` du paquet
// Les autres fonctions et variables du paquet seront inaccessibles
#import "@preview/community-cnam-thesis:0.1.0": info-box, warning-box
```

La première fois que vous importez un paquet Typst dans votre fichier `*.typ`, le compilateur Typst télécharge automatiquement le paquet depuis le dépôt GitHub et le met en cache, ce qui permet de l'utiliser ultérieurement sans avoir à le télécharger à nouveau.

Vous l'aurez compris, il n'existe pas de gestionnaire de paquets au sens classique pour gérer les dépendances. Vous pouvez bien sûr gérer vos dépendances manuellement en modifiant les versions des paquets dans vos fichiers Typst, mais cela peut devenir fastidieux et source d'erreurs. C'est pourquoi, il est recommandé d'utiliser un gestionnaire de paquets pour gérer vos dépendances. Les fonctionnalités minimales attendues d'un gestionnaire de paquets pour Typst sont les suivantes :

- Initialiser un projet pour créer la bonne structure de fichiers et de dossiers pour un projet Typst ;
- Copier les fichiers dans un espace de nommage donné (typiquement `@local` ou `@preview`) ;
- Synchroniser les dépendances avec le dépôt GitHub des paquets Typst pour mettre à jour les versions des paquets.

[3] Si vous souhaitez rechercher en fonction du nombre d'étoiles sur GitHub, vous pouvez utiliser Typst Universe with Stars (Unofficial)

Parmi les gestionnaires existants répondant (même partiellement) à ces critères, on peut citer :

- UTPM – Unofficial Typst Package Manager (mon préféré 😉)
- GoTPM – Go Typst Package Manager
- Tyler et Typship

Remarque

Si vous avez simplement besoin de copier vos fichiers Typst dans un espace de nommage donné, vous pouvez utiliser les scripts `just` que vous trouverez dans le dépôt GitHub Typst Package Template.

Chapitre

1

Usage général

Ce chapitre présente les instructions générales pour l'utilisation du gabarit `community-cnam-thesis`. Il est recommandé de suivre ces instructions avant de commencer la rédaction de votre manuscrit.

Contenu

1.1. Informations générales 22
1.2. Initialisation du gabarit 22
1.3. Arborescence des fichiers 26
1.4. Structure du fichier principal 26

1.1. Informations générales

Le gabarit `community-cnam-thesis` est basé sur `bookly`, qui est paquet Typst développé par l'auteur de ce document. Il fournit une structure de document cohérente, des styles de mise en page prédéfinis et des fonctionnalités spécifiques aux documents académiques.

Le présent gabarit personnalise le gabarit `bookly` pour répondre aux besoins spécifiques des thèses du Conservatoire national des arts et métiers (Cnam). Il inclut d'autres fonctionnalités qui seront détaillées dans les chapitres suivants.

1.1.1. Polices de caractères

Pour utiliser le gabarit `community-cnam-thesis`, il est nécessaire d'installer les polices de caractères suivantes sur votre système :

- Texte : `TeXGyrePagellaX` (lien de téléchargement), `Libertinus Serif` (lien de téléchargement) et `New Computer Modern` (inclus avec Typst).
- Mathématiques : `TeX Gyre Pagella Math` (lien de téléchargement), `Libertinus Math` (lien de téléchargement) et `New Computer Modern Math` (inclus avec Typst).
- Code : `Cascadia Code` (lien de téléchargement), `Hack` (lien de téléchargement) et `DejaVu Sans Mono` (inclus avec Typst).

1.1.2. Couleurs du thème

Le gabarit `community-cnam-thesis` définit deux couleurs principales pour assurer une cohérence visuelle dans l'ensemble du document :

- Couleur primaire
- Couleur secondaire :

Remarque

Les couleurs sont définies dans le `dictionary cnam-colors`. Vous pouvez y accéder en utilisant la syntaxe classique de Typst : `cnam-colors.primary` et `cnam-colors.secondary`.

1.2. Initialisation du gabarit

Pour utiliser le modèle, il faut l'importer dans votre fichier principal `typ`. En supposant que le template et le fichier principal sont dans le même dossier, il suffit d'insérer la commande suivante à la première ligne du fichier principal.

Code

```
#import "@local/community-cnam-thesis:0.1.4": *
```

Remarque

Si vous décomposez votre document en différents fichiers, il faut insérer la commande précédente en préambule de chaque fichier.

Après avoir importé le modèle, celui doit être initialisé en appliquant une règle d'affichage (`show` rule) avec la commande `#community-cnam-thesis()` en passant les options nécessaires avec l'instruction `with` dans votre fichier principal `typ` :

Code

```
#show: community-cnam-thesis.with(
    title: "titre de la thèse",
    author: "Nom de l'auteur",
    lang: "fr",
    open-right: true,
    thesis-info: thesis-info-default,
)
```

Cette fonction d'initialisation contient un certain nombre d'arguments qui sont détaillés ci-dessous. Il est possible de modifier ces arguments en fonction de vos besoins.

Paramètre

`⟨title⟩: "Nom de la thèse"` `string` | `content`

Titre de la thèse

Paramètre

`⟨author⟩: "Nom de l'auteur"` `string` | `content`

Auteur de la thèse

Paramètre

`⟨lang⟩: "fr"` `string`

Langue du document.

Remarque

Toutes les langues supportées par `bookly` sont supportées par le gabarit `community-cnam-thesis`.

Paramètre

`⟨open-right⟩: true` `bool`

Si `true`, les chapitres s'ouvrent sur une page de droite. Si `false`, les chapitres s'ouvrent sur la page suivante.

Paramètre

`(thesis-info)`: `thesis-info-default` `dictionary`

Dictionnaire contenant les informations relatives à la thèse.

- `doctoral-school` `string` | `content` : École doctorale de rattachement
- `supervisor` `array` : Définition du ou des directeurs de thèse. Chaque membre est défini par un `dictionary` contenant les clés suivantes:
  - `name` `string` | `content` : Nom du directeur
  - `title` `string` | `content` : Statut du directeur (MCF HDR, PU, PRCM, …)
  - `institution` `string` | `content` : Établissement/Entreprise de rattachement du directeur,
- `supervisor` `array` : Définition du ou des co-encadrants de thèse. Chaque membre est défini par un `dictionary` contenant les clés suivantes:
  - `name` `string` | `content` : Nom du co-encadrant
  - `title` `string` | `content` : Statut du co-encadrant (MCF, MCF HDR, PU, PRCM, …)
  - `institution` `string` | `content` : Établissement/Entreprise de rattachement du co-encadrant,
- `laboratory` `string` | `content` : Nom du laboratoire de rattachement
- `defense-date` `string` | `content` : Date de soutenance de la thèse
- `discipline` `string` | `content` : Discipline de la thèse
- `speciality` `string` | `content` : Spécialité de la thèse
- `committee` `array` : Membres du jury de soutenance. Chaque membre est défini par un `dictionary` contenant les clés suivantes:
  - `name` `string` | `content` : Nom du membre du jury
  - `position` `string` | `content` : Poste du membre du jury
  - `role` `string` | `content` : Rôle du membre du jury (Président, Rapporteur, Examinateur, etc.)
- `dedication` `string` | `content` : Dédicace de la thèse
- `logo` `array` : Chemin vers le logo de l'institution

Valeurs par défaut

- `doctoral-school`: "Sciences des Métiers de l'Ingénieur",
- supervisor: `(:)`,
- co-supervisor: `(:)`,
- laboratory: `none`,
- defense-date: `none`,
- discipline: "Sciences de l'ingénieur",

- speciality: "Mécanique",
- committee: (:),
- dedication: none,
- logo: none

Pour définir les dictionnaires `supervisor`, `co-supervisor` et `committee`, plusieurs approches sont possibles :

1. Définition directe en Typst.

Code

```
// main.typ
#let supervisor = (
  (name: "Henri Grégoire", title: "Abbé constitutionnelle", institution: "Cnam,
Paris"),
  (name: "Henri Tresca", title: "Professeur titulaire de la Chaire de Mécanique",
institution: "Cnam, Paris"),
)

#show: community-cnam-thesis.with(
    thesis-info: (
        supervisor: supervisor,
    ),
)
```

2. Définition dans un fichier `json`.

Code

```
// thesis-info.json
{
  "supervisor": [
    {
      "name": "Henri Grégoire",
      "title": "Abbé constitutionnelle",
      "institution": "Cnam, Paris"
    },
    {
      "name": "Henri Tresca",
      "title": "Professeur titulaire de la Chaire de Mécanique",
      "institution": "Cnam, Paris"
    }
  ]
}

// main.typ
#show: community-cnam-thesis.with(
    thesis-info: json("/chemin/vers/thesis-info.json"),
)
```

3. Définition dans un fichier `yaml` séparé.

Code

```yaml
# thesis-info.yaml
supervisor:
  - name: "Henri Grégoire"
    title: "Abbé constitutionnelle"
    institution: "Cnam, Paris"
  - name: "Henri Tresca"
    title: "Professeur titulaire de la Chaire de Mécanique"
    institution: "Cnam, Paris"
```

```typst
// main.typ
#show: community-cnam-thesis.with(
    thesis-info: yaml("/chemin/vers/thesis-info.yaml"),
)
```

1.3. Arborescence des fichiers

Lorsque l'on rédige un document long, comme peut l'être un manuscrit de thèse, il est préférable d'adopter une organisation multi-fichiers, afin d'éviter d'avoir un looooooong fichier principal. Cela est notamment important lors de la phase de relecture/correction, ainsi que dans le cadre d'une écriture collaborative. Pour ma part, j'adopte généralement la structure suivante:

```
main.typ
├─ annexes/
│  └─ annexe1.typ
├─ bibliographie/
│  └─ biblio.bib
├─ chapitres/
│  └─ chapitre1.typ
├─ images/
│  └─ logo.png
└─ preambule/
   └─ resume.typ
```

1.4. Structure du fichier principal

À partir de la structure définie dans la section précédente, le fichier principal `main.typ` pourrait ressembler à ceci[1] :

[1] Le code ci-dessous est le fichier principal utiliser pour rédiger le présent document.

Code

```
// main.typ
#import "@local/community-cnam-thesis:0.1.4": *

#let supervisor = ...
#let co-supervisor = ...
#let committee = ...

#show: community-cnam-thesis.with(
   title: [Guide d'utilisation du template \ Typst pour les thèses du Cnam],
    author: "Mathieu Aucejo",
    thesis-info: (
        doctoral-school: "Sciences des Métiers de l'Ingénieur",
        supervisor: supervisor,
        laboratory: "LMSSC",
        co-supervisor: co-supervisor,
        defense-date: "15 juin 2024",
        discipline: "Sciences de l'ingénieur",
        speciality: "Mécanique",
        committee: committee,
        dedication: [Les logiciels et les cathédrales, c'est un peu la même chose,
d'abord on les construit, ensuite on prie. \ Sam Redwine],
    ),
    lang: "fr",
    open-right: true
)

#show: front-matter

#include "avant-propos/avant-propos.typ"

#show: main-matter

#tableofcontents
#listoffigures
#listoftables

#part[Guide d'utilisation]

#include "chapitres/chapitre-main.typ"

#bibliography("bibliographie/biblio.bib")

#show: appendix

#part[Annexes]
#include "annexes/annexes-main.typ"

#backcover(resume: ..., abstract: ...)
```

Le fichier principal présente la structure générale du document. On peut remarquer que le document comporte :

- Des environnements de préambule **`#front-matter`**, de corps de texte **`#main-matter`** et d'annexes **`#appendix`**.

- Une table des matières `#tableofcontents`, une liste de figures `#listoffigures` et une liste de tableaux `#listoftables`.
- Des parties `#part`.
- Des chapitres et annexes importés depuis d'autres fichiers grâce à la commande `#include`.
- Une bibliographie insérée avec la commande `#bibliography()`.
- Une quatrième de couverture `#backcover` contenant le résumé et l'abstract du document.

Tous ces éléments sont détaillés dans les chapitres suivants.

Chapitre

2

Mise en forme du document

Ce chapitre présente les éléments de base permettant de mettre en forme un document Typst. On évoquera en particulier de la hiérarchisation des titres, du référencement des éléments et des différents éléments de mise en forme du contenu.

Contenu

2.1. Hiérarchisation des titres 30
2.2. Références croisées 31
2.3. Mise en forme du texte 31
2.4. Parties et environnements de mise en page 33
2.5. Quatrième de couverture 34

2.1. Hiérarchisation des titres

Le template `community-cnam-thesis` supporte différents niveaux de titres.

Code

```
= Titre de chapitre (niveau 1)

== Titre de section (niveau 2)

=== Titre de sous-section (niveau 3)

==== Titre de sous-sous-section (niveau 4)
```

Attention

Un plan à quatre niveau numérotés est généralement le maximum acceptable dans une thèse. Si vous avez besoin d'un plan plus profond, il est recommandé de revoir la structure de votre manuscrit.

2.1.1. Chapitres numérotés et non numérotés

Par défaut, les chapitres sont numérotés. Si vous souhaitez créer des chapitres non numérotés (résumé, remerciements, introduction, etc.), vous pouvez utiliser la commande suivante en préambule du fichier correspondant. Par exemple, pour le chapitre d'introduction:

Code

```
#import "@local/community-cnam-thesis:0.1.4": *

#show: chapter-nonum

= Titre du chapitre
```

2.1.2. Sections numérotées et non numérotées

Par défaut, tous les titres de niveau 1 à 4 sont numérotés. Pour insérer des sections ou sous-sections non-numérotés, il faut ajouter attaché le `label` `<nonum-sec>` à la fin du titre de la section ou de la sous-section concernée. Par exemple:

Code

```
== Titre de section non numérotée <nonum-sec>
=== Titre de sous-section non numérotée <nonum-sec>
```

2.2. Références croisées

Pour créer des références croisées vers des sections, figures, tableaux, équations, etc., il faut d'abord définir un label pour l'élément que l'on souhaite référencer. Cela se fait en ajoutant le `label <mon-label>` à la fin de l'élément concerné :

Code

```
= Titre du chapitre <ch:chapitre>

== Titre de section <s:section>
```

Pour citer des éléments référencés, on utilise la commande `@mon-label`.

typ
```
Pour plus de détails, voir le chapitre
@ch:structure et la section @s:structure-
titles.
```

Rendu

Pour plus de détails, voir le chapitre 2 et la section 2.1.

2.3. Mise en forme du texte

Cette section présente différents éléments de mise en forme du texte.

2.3.1. Emphase et code

typ
```
*gras*, _italique_, _*gras et italique*_,
`code inline`, #strike[barré],
#super[exposant], #sub[indice]
```

Rendu

gras, *italique*, ***gras et italique***, `code inline`, ~~barré~~, ^{exposant}, $_{\text{indice}}$

2.3.2. Liens hypertextes

Pour créer des liens hypertextes, il faut utiliser la commande `#link()` de Typst (voir Documentation Typst pour plus de détails).

typst
```
#link("https://www.cnam.fr", "Le site du
Cnam")

https://www.cnam.fr

#link("mailto:contact@cnam.fr")
```

Rendu

Le site du Cnam

https://www.cnam.fr

contact@cnam.fr

2.3.3. Listes

Il existe plusieurs types de listes dans Typst: les listes à puces, les listes numérotées et les listes de tâches.

typ
```
- Premier élément
- Deuxième élément
  - Sous-élément 1 (1 tabulation)
```

Rendu

- Premier élément

```
+ Premier élément
+ Deuxième élément
  + Premier élément

#sym.dots.v
5. Cinquième élément
+ Sixième élément

- [x] Tâche 1
- [ ] Tâche 2
```

- Deuxième élément
  - Sous-élément 1 (1 tabulation)

1. Premier élément
2. Deuxième élément
   a. Premier élément

   ⋮
5. Cinquième élément
6. Sixième élément

- [x] Tâche 1
- [ ] Tâche 2

⚠ Attention

Les listes de tâches sont adaptées pour les documents de type « guide » ou « manuel », mais ne sont pas adaptées pour un manuscrit de thèse. Il est recommandé d'utiliser des listes à puces ou numérotées pour la rédaction d'une thèse. Cela est plus neutre et conforme aux standards académiques.

2.3.4. Citations

Les citations sont utilisées pour référencer des œuvres ou des idées développées par d'autres auteurs.

typst
```
#quote(attribution: "Platon", block:
true)[Il y a en chacun de nous des
calculs que nous nommons espérance.]
```

Rendu

> Il y a en chacun de nous des calculs que nous nommons espérance.
>
> — Platon

2.3.5. Notes de bas de pages

typst
```
 Le Cnam a été fondé le 10 octobre
1794#footnote[19 vendémiaire an III de la
Révolution.].
```

Rendu

Le Cnam a été fondé le 10 octobre 1794[1].

2.3.6. Sauts de page

Pour insérer un saut de page, il faut utiliser la commande `#pagebreak()` de Typst (voir Documentation Typst pour plus de détails).

</> Code

```
Fin du contenu de cette section.
```

[1] 19 vendémiaire an III de la Révolution.

```
#pagebreak()

Début du contenu de la section suivante.
```

⚠ Attention

Il est recommandé d'utiliser les sauts de page avec parcimonie, afin de ne pas perturber la lecture du document. Il est préférable de laisser le moteur de mise en page gérer les sauts de page automatiquement. Préférez les ajuster en toute fin de rédaction, une fois le contenu figé.

2.4. Parties et environnements de mise en page

Les parties sont des sections de niveau supérieur aux chapitres. Elles permettent de regrouper plusieurs chapitres sous un même thème. Pour créer une partie, il faut utiliser la commande `#part()`.

Code

```
// Création d'une partie
#part[Titre de la partie]
```

Concernant les environnements de mise en page, `community-cnam-thesis` s'appuie sur le template `bookly` pour structurer le document. Il est possible d'utiliser les environnements `#front-matter`, `#main-matter` et `#appendix` pour structurer le document en trois parties : la partie avant-propos, la partie principale et la partie annexes.

Pour activer ces environnements, il faut utiliser les commandes suivantes à l'endroit souhaité dans le document :

Code

```
#show: front-matter

// Contenu de la partie avant-propos

#show: main-matter

// Contenu de la partie principale

#show: appendix

// Contenu de la partie annexes
```

2.5. Quatrième de couverture

La quatrième de couverture est une page qui présente le résumé et l'abstract du document. Elle est généralement utilisée pour donner un aperçu du contenu du document aux lecteurs.

Code

```
#backcover(resume: ..., abstract: ...)
```

La fonction `#backcover()` accepte les arguments suivants :

Paramètre

`⟨resume⟩: none` — `string` | `content`

Résumé en français du document

Paramètre

`⟨abstract⟩: none` — `string` | `content`

Abstract en anglais du document

Chapitre

3

Figures et tableaux

Ce chapitre présente les différentes commandes et environnements disponibles pour insérer des figures et des tableaux dans le document.

Contenu

3.1. Figures 36
3.2. Tableaux 37
3.3. Titres longs et courts pour les figures et tableau 39
3.4. Création de courbes directement en Typst 39

3.1. Figures

La syntaxe de base pour insérer une image est la suivante[1]:

Code

```
#image("chemin/vers/image.png", width: 10cm)
```

typst

```
#image("./images/cnam.png")
```

Rendu



Remarque

Typst accepte un certain nombre de formats d'image. Les formats actuellement supportés sont les suivants : PNG, JPEG, GIF, SVG, PDF, WEBP et Raw Pixel Data.

Cependant, lors de la rédaction d'un texte scientifique, il est souvent nécessaire d'ajouter une légende à l'image et de la référencer dans le texte. Pour cela, il est recommandé d'utiliser l'environnement `#figure()` de Typst. La syntaxe est la suivante[2]:

Code

```
#figure(
  image("chemin/vers/image.png", width: 10cm),
  caption: "Légende de l'image"
) <fig:mon-label>
```

Cette commande permet d'insérer une image avec une légende et un label pour la référence croisée. Le label est défini à la fin de l'environnement `#figure()` avec la syntaxe `<mon-label>`. Pour citer cette figure dans le texte, il suffit d'utiliser la commande `@mon-label`.

typst

```
#figure(
  image("./images/cnam.png"),
  caption: "Logo officiel du Cnam"
) <fig:logo-cnam>

La Figure @fig:logo-cnam présente le logo
officiel du Cnam.
```

Rendu



Figure 3.1 – Logo officiel du Cnam

La Figure 3.1 présente le logo officiel du Cnam.

[1] Pour plus d'informations sur la commande `#image()`, voir Documentation Typst
[2] Pour plus d'informations sur l'environnement `#figure()`, voir Documentation Typst

On peut également insérer plusieurs images dans une seule figure en utilisant l'environnement `#subfigure()` du template `bookly` sur lequel est basé le présent gabarit. La syntaxe est la suivante :

Code

```
#subfigure(
  figure(image("../images/image1.png"), caption: []),
  figure(image("../images/image2.png"), caption: []), <fig:b>,
  columns: (1fr, 1fr),
  caption: [(a) Left image and (b) Right image],
  label: <fig:subfig>,
)
```

typ

```
#subfigure(
  figure(image("./images/cnam.png"), caption: []),
  figure(image("./images/victoire.svg", width: 50%), caption: []), <fig:b>,
  columns: (1fr, 1fr),
  caption: [(a) Logo du Cnam and (b) Estampille de la Victoire],
  label: <fig:subfig>,
)

La Figure @fig:subfig présente le logo officiel du Cnam et l'allégorie de la Victoire
ailée, associée à la devise latine "_Docet Omnes Ubique_" (voir Figure @fig:b).
```

Rendu

le cnam



(a) (b)

Figure 3.2 – (a) Logo du Cnam and (b) Estampille de la Victoire

La Figure 3.2 présente le logo officiel du Cnam et l'allégorie de la Victoire ailée, associée à la devise latine « *Docet Omnes Ubique* » (voir Figure 3.2b).

3.2. Tableaux

Les tableaux peuvent être insérés dans le document en utilisant l'environnement `#table()` de Typst. La syntaxe est la suivante[3]:

[3]Pour plus d'informations sur l'environnement `#table()`, voir Documentation Typst

Code

```
#table(
  columns: 3,
  align: horizon,
  [*Colonne 1*], [*Colonne 2*], [*Colonne 3*],
  [Ligne 1, Cellule 1], [Ligne 1, Cellule 2], [Ligne 1, Cellule 3],
  [Ligne 2, Cellule 1], [Ligne 2, Cellule 2], [Ligne 2, Cellule 3],
)
```

typst

```
#table(
    columns: (1fr,)*2,
    align: center + horizon,
    inset: 5pt,
    [*Forme*], [*Périmètre*],
    [Carré de côté $a$],[$4a$],
    [Cercle de rayon $r$],[$2pi r$],
  )
```

Rendu

Forme	Périmètre
Carré de côté a	$4a$
Cercle de rayon r	$2\pi r$

Comme pour les figures, il est possible d'ajouter une légende et un label à un tableau en utilisant l'environnement `#table()` du template `bookly`. La syntaxe est la suivante :

Code

```
#figure(
  table(
    columns: 3,
    align: horizon,
    [*Colonne 1*], [*Colonne 2*], [*Colonne 3*],
    [Ligne 1, Cellule 1], [Ligne 1, Cellule 2], [Ligne 1, Cellule 3],
    [Ligne 2, Cellule 1], [Ligne 2, Cellule 2], [Ligne 2, Cellule 3],
  ),
  caption: "Légende du tableau",
)
```

typ

```
#let mon-tableau = table(
    columns: 2,
    align: center + horizon,
    inset: 5pt,
    [*Forme*], [*Périmètre*],
    [Carré de côté $a$],[$4a$],
    [Cercle de rayon $r$],[$2pi r$],
  )

#figure(
  mon-tableau,
  caption: "Tableau des périmètres de
quelques formes géométriques",
) <tab:perimetres>
```

Rendu

Tableau 3.1 – Tableau des périmètres de quelques formes géométriques

Forme	Périmètre
Carré de côté a	$4a$
Cercle de rayon r	$2\pi r$

Le Tableau 3.1 présente les formules de calcul des périmètres de quelques formes géométriques.

```
Le Tableau @tab:perimetres présente les
formules de calcul des périmètres de
quelques formes géométriques.
```

Remarque

Le lecteur attentif aura noté que pour éviter d'alléger le code, nous avons défini le tableau dans une variable `mon-tableau` avant de l'insérer dans l'environnement `#figure()`. Cela permet de réutiliser le même tableau à différents endroits du document si nécessaire et de rendre le code plus lisible.

3.3. Titres longs et courts pour les figures et tableau

Le template `community-cnam-thesis` permet de définir un titre long et un titre court pour les figures et les tableaux via la fonction `#short-or-long()`. Si appliqué aux titres de chapitre ou de section, le titre long est utilisé dans le titre lui-même, tandis que le titre court est utilisé dans l'en-tête de page. Si utilisé pour les figures ou les tableaux, le titre long est utilisé dans la légende de la figure ou du tableau, tandis que le titre court est utilisé dans la liste des figures ou des tableaux.

typst
```
#figure(
    rect(),
    caption: short-or-long[Représentation
d'un rectangle][Ceci est une figure
représentant un rectangle (voir la table
des figures pour voir le titre court)]
  )
```

Rendu

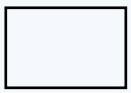


Figure 3.3 – Ceci est une figure représentant un rectangle (voir la table des figures pour voir le titre court)

3.4. Création de courbes directement en Typst

Typst étant un langage Turing complet, il est possible de créer des courbes directement dans le document en utilisant par exemple le package `lilaq`. Ceci permet de créer des graphiques et des visualisations directement dans le document sans avoir besoin d'importer des images externes et d'avoir une mise en page cohérente avec le reste du document.

L'exemple ci-dessous montre comment créer une figure avec deux courbes représentant la réponse libre d'un système mécanique à un degré de liberté non amorti et sous-amorti. Pour plus d'informations sur le package `lilaq`, voir la Documentation de lilaq.

typ
```
#import "@preview/lilaq:0.6.0" as lq

// Fonction calculant la réponse libre d'un système mécanique à un degré de liberté
#let free_response(om0, xi, x0, v0, t) = {
  if xi < 1. {
    // Sous-amorti
    let Om0 = om0 * calc.sqrt(1 - xi * xi)
```

```
    let a = x0
    let b = (v0 + xi * om0 * x0) / Om0
    calc.exp(-xi * om0 * t) * (a * calc.cos(Om0 * t) + b * calc.sin(Om0 * t))
  } else if xi ≤ 1. {
    // Amortissement critique
    (x0 + (v0 + om0 * x0) * t) * calc.exp(-om0 * t)
  } else {
    // Sur-amorti
    let s = om0*calc.sqrt(xi * xi - 1)
    let a = x0
    let b = (v0 + xi * om0 * x0)/s
    calc.exp(-xi * om0 * t) * (a * calc.cosh(s * t) + b * calc.sinh(s * t))
  }
}

#let om0 = 2 * calc.pi * 2            // Pulsation propre du système (rad/s)
#let x0 = 0.02                        // Déplacement initial (m)
#let v0 = -0.5                        // Vitesse initiale (m/s)
#let t = lq.linspace(0, 5, num: 500)  // Vecteur temps (s)

// Création de la courbe de réponse libre non amortie
#let undamped = lq.diagram(
  width: 100%,
  title: [*Réponse libre non amortie*],
  xlabel: [Temps (s)],
  ylabel: [Déplacement (m)],
  margin: 0%,
  lq.plot(t, t => free_response(om0, 0, x0, v0, t),
  color: cnam-colors.primary,
  mark: none,
  ),
)

// Création de la courbe de réponse libre sous-amortie
#let underdamped = lq.diagram(
  width: 100%,
  title: [Réponse libre sous-amortie],
  xlabel: [Temps (s)],
  margin: 0%,
  lq.plot(t, t => free_response(om0, 0.1, x0, v0, t),
  color: cnam-colors.primary,
  mark: none,
  ),
)

// Création de la grille pour afficher les deux courbes côte à côte
#let fig-grid = grid(
  columns: (1fr,)*2,
  column-gutter: 1em,
  [#undamped], [#underdamped],
)

// Création de la figure
#figure(
  fig-grid,
  caption: [Réponse libre d'un système mécanique à un degré de liberté],
) <fig:free-response>
```

```
La Figure @fig:free-response présente la réponse libre d'un système mécanique à un
degré de liberté non amorti et sous-amorti.
```

Rendu

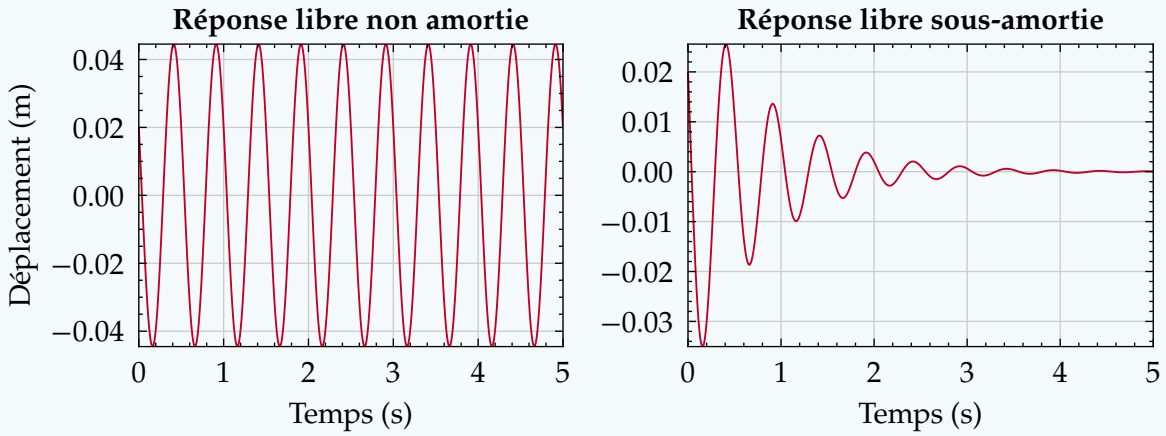


Figure 3.4 – Réponse libre d'un système mécanique à un degré de liberté

La Figure 3.4 présente la réponse libre d'un système mécanique à un degré de liberté non amorti et sous-amorti.

Chapitre

4

Équations et algorithmes

Ce chapitre présente les différentes manières d'écrire des équations et d'insérer des algorithmes dans un document de thèse.

Contenu

4.1. Équations 44
4.2. Algorithmes 46

4.1. Équations

⚠ Attention

La syntaxe des équations Typst diffère de celle de LaTeX. Pour plus d'informations sur la syntaxe des équations Typst, veuillez consulter la documentation officielle de Typst.

Pour accéder à la liste complète des symboles mathématiques disponibles dans Typst, c'est par ici.

Pour la liste complète des emojis (et oui, c'est possible 😻), c'est par là. En revanche, leur utilisation est à proscrire dans un document académique (mais je trouve ça amusant à signaler).

4.1.1. Équations en ligne

Les équations en ligne s'écrivent entre les symboles `$ … $` et sont intégrées dans le texte.

typ
```
#let st = sym.space.thin

La transformée de Fourier d'une fonction
$f(t)$ est définie par l'intégrale
$hat(f)(xi) = integral_(-oo)^(+oo) f(t)
st e^(-2 pi i xi t) dif t$
```

Rendu

La transformée de Fourier d'une fonction $f(t)$ est définie par l'intégrale $\hat{f}(\xi) = \int_{-\infty}^{+\infty} f(t)\, e^{-2\pi i \xi t}\, \mathrm{d}t$

4.1.2. Équations numérotées et non numérotées

Les équations numérotées s'écrivent entre les symboles `$ … $` et sont centrées sur la page. Comme pour les figures et les tableaux, on peut ajouter un `label` pour pouvoir y faire référence dans le texte.

⚠ Attention

La différence entre les équations numérotées et les équations en ligne est subtile. En effet, une équation en ligne s'écrira :

`$y = f(x)$`,

tandis qu'une équation numérotée s'écrira (notez l'espace avant et après le symbole `$`):

`$ y = f(x) $`.

typ
```
$
i planck (partial Psi)/ (partial t) =
hat(H) space.thin Psi.
$ <eq:schrodinger>

L'équation de Schrödinger @eq:schrodinger
décrit l'évolution temporelle #sym.dots
```

Rendu

$$i\hbar\frac{\partial \Psi}{\partial t} = \hat{H}\,\Psi. \tag{4.1}$$

L'équation de Schrödinger (4.1) décrit l'évolution temporelle …

Pour ne pas numéroter une équation, il suffit d'ajouter le `label <nonum-eq>` après le symbole `$` de fermeture de l'équation. Par exemple, l'équation suivante ne sera pas numérotée :

typ
```
$
i planck (partial Psi)/ (partial t) =
hat(H) space.thin Psi.
$ <nonum-eq>
```

Rendu

$$i\hbar\frac{\partial\Psi}{\partial t} = \hat{H}\,\Psi.$$

4.1.3. Environnements avancés

Les équations peuvent également être écrites dans des environnements plus complexes, tels que les systèmes d'équation. Par exemple, pour écrrire des équations sur plusieurs lignes, on peut écrire:

typ
```
#let bm(x) = $upright(bold(#x))$
$
  nabla dot.c bm(E) &= rho/epsilon_0, \
  nabla dot.c bm(B) &= 0, \
  nabla times bm(E) &= - (partial bm(B))/
(partial t), \
  nabla times bm(B) &= mu_0 space.thin
bm(J) + mu_0 epsilon_0 (partial bm(E))/
(partial t).
$
```

Rendu

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}, \tag{4.2a}$$
$$\nabla \cdot \mathbf{B} = 0, \tag{4.2b}$$
$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}, \tag{4.2c}$$
$$\nabla \times \mathbf{B} = \mu_0\,\mathbf{J} + \mu_0\varepsilon_0\frac{\partial \mathbf{E}}{\partial t}. \tag{4.2d}$$

On peut également écrire les équations précédentes sous la forme d'une seul équation numérotée:

typ
```
#let bm(x) = $upright(bold(#x))$
$
  nabla dot.c bm(E) &= rho/epsilon_0, \
  nabla dot.c bm(B) &= 0, \
  nabla times bm(E) &= - (partial bm(B))/
(partial t), \
  nabla times bm(B) &= mu_0 space.thin
bm(J) + mu_0 epsilon_0 (partial bm(E))/
(partial t).
$ <equate:revoke>
```

Rendu

$$\begin{aligned}
\nabla \cdot \mathbf{E} &= \frac{\rho}{\varepsilon_0},\\
\nabla \cdot \mathbf{B} &= 0,\\
\nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t},\\
\nabla \times \mathbf{B} &= \mu_0\,\mathbf{J} + \mu_0\varepsilon_0\frac{\partial \mathbf{E}}{\partial t}.
\end{aligned} \tag{4.3}$$

Remarque

La numérotation des sous-équations est gérée par le paquet `equate`. Le `label <equate:revoke>` permet d'utiliser le système de numérotation natif de Typst.

On peut également écrire les équations sous la forme d'un système d'équations, en procédant comme suit :

typ
```
$
  cases(
    dot(x)(t) &= A x(t) + B u(t),
    y(t) &= C x(t) + D u(t),
  )
$
```

Rendu

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) + Du(t) \end{cases} \tag{4.4}$$

Le template `bookly` permet la création d'équations encadrées en utilisant la commande `#boxeq()`.

typ
```
$
  #boxeq($y = f(x)$)
$
```

Rendu

$$\boxed{y = f(x)} \tag{4.5}$$

Enfin, il est possible d'ajouter de la couleur dans les équations :

typ
```
#let colred(x) = text(fill: cnam-
colors.primary, $#x$)
#let colblue(x) = text(fill: blue, $#x$)

$
  colred(y) = colblue(f)(x)
$
```

Rendu

$$y = f(x) \tag{4.6}$$

4.2. Algorithmes

Les algorithmes peuvent être insérés dans le document de thèse en utilisant la `#algorithm()` qui est basée sur le package `lovelace`. Cette commande accepte les paramètres suivants :

Paramètre

`⟨caption⟩: none` — `string` | `content`

Légende de l'algorithme. Si `none`, aucune légende ne sera affichée.

Paramètre

`⟨line-numbering⟩: "1"` — `string`

Numérotation des lignes de l'algorithme.

Si `none`, aucune numérotation ne sera affichée.

typ

```
#algorithm(caption: [Mon algorithme])[
  + do something
  + *while* still something to do
    + *if* not done yet *then*
      + wait a bit
      + resume working
    + *else*
      + go home
    + *end*
  + *end*
] <alg:example>

#noindent L'algorithme @alg:example est
un exemple d'algorithme simple
complètement inutile.
```

Rendu

Algorithme 4.1 – Mon algorithme

```
1  do something
2  while still something to
   do
3  | do even more
4  | if not done yet then
5  | | wait a bit
6  | | resume working
7  | else
8  | | go home
9  | end
10 end
```

L'algorithme 4.1 est un exemple d'algorithme simple complètement inutile.

Chapitre

5

Boîtes informationnelles

Les boîtes informationnelles sont des éléments de mise en forme qui permettent de mettre en avant certaines informations dans un document. Elles sont souvent utilisées pour attirer l'attention du lecteur sur des points importants, des conseils, des avertissements ou des notes supplémentaires.

Contenu

5.1. Boîtes disponibles 50
5.2. Cas particuliers des boîtes de code 51

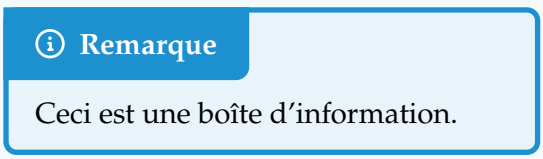
5.1. Boîtes disponibles

Le template `community-cnam-thesis` propose plusieurs types de boîtes informationnelles, chacune ayant un style et une fonction spécifique. Voici les principales boîtes disponibles :

- `#info-box` : Utilisée pour fournir des informations générales ou des conseils utiles.

```typ
#info-box[Ceci est une boîte d'information.]
```

Rendu



- `#tip-box` : Utilisée pour donner des astuces ou des recommandations.

```typ
#tip-box[Ceci est une boîte de conseil.]
```

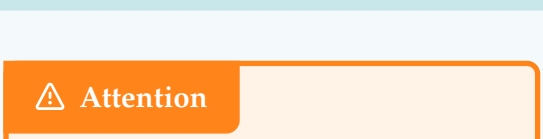
Rendu



- `#warning-box` : Utilisée pour avertir le lecteur d'un danger potentiel ou d'une information importante.

```typ
#warning-box[Ceci est une boîte d'avertissement.]
```

Rendu



- `#important-box` : Utilisée pour souligner des informations cruciales ou des points essentiels.

```typ
#important-box[Ceci est une boîte importante.]
```

Rendu



- `#proof-box` : Utilisée pour présenter des preuves ou des démonstrations mathématiques.

typ
```
#proof-box[Ceci est une boîte de preuve.]
```

Rendu

Démonstration

Ceci est une boîte de preuve.

- `#question-box` : Utilisée pour poser des questions ou des problèmes à résoudre.

typ
```
#question-box[Ceci est une boîte de
question.]
```

Rendu

Question

Ceci est une boîte de question.

- `#code-box` : Utilisée pour présenter du code source ou des extraits de code.

typ
```
#code-box[Ceci est une boîte de code.]
```

Rendu

Code

Ceci est une boîte de code.

Attention

Les boîtes informationnelles doivent être utilisées de manière judicieuse et avec parcimonie pour ne pas surcharger le document.

5.2. Cas particuliers des boîtes de code

Pour remplir une boîte de de code, il est possible d'utiliser la syntaxe native de Typst :

typ
````
#code-box[```julia
import LinearAlgebra

A = rand(3, 3)
b = rand(3)

x = A \ b
```]
````

Rendu

Code

```julia
import LinearAlgebra

A = rand(3, 3)
b = rand(3)
```
```

```
x = A \ b
```

On peut rajouter la numérotation des lignes de code en utilisant des paquets dédiés, comme par exemple `zebraw`:

typ

````
#code-box[#zebraw(
  lang: false,
  ```julia
  import LinearAlgebra

  A = rand(3, 3)
  b = rand(3)

  x = A \ b
  ```
)]
````

Rendu

Code

```
1 import LinearAlgebra
2
3 A = rand(3, 3)
4 b = rand(3)
5
6 x = A \ b
```

Remarque

Le paquet `zebraw` est une dépendance explicite du template `community-cnam-thesis`. Il peut donc être utilisé directement sans l'importer de manière explicite.

Pour aller plus loin, vous pouvez consulter la documentation de `zebraw`. On peut noter que d'autre paquets existent pour la mise en forme de code, comme `codly` ou `codelst`. En revanche, ces paquets ne sont pas des dépendances du template `community-cnam-thesis` et doivent donc être importés explicitement si vous souhaitez les utiliser.

Chapitre

6

Références

Ce chapitre présente le système de citations bibliographiques utilisé dans le template de thèse du Cnam. Il est basé sur le style de citation IEEEtran, adapté pour le français. Le fichier de style utilisé est `IEEEtran-francais.csl`, qui est inclus dans le gabarit `community-cnam-thesis`. Le système de glossaire et d'acronyme est également présenté.

Contenu

6.1. Références bibliographiques 54
6.2. Glossaire, acronymes et nomenclatures 55

6.1. Références bibliographiques

Cette section explique comment insérer des citations dans votre document.

6.1.1. Format de fichiers

Typst accepte nativement les fichiers de bibliographie au format BibTeX. Toutefois, Typst introduit un nouveau format de fichier de bibliographie appelé Hayagriva, qui est un fichier `yaml` contenant les références bibliographiques. Pour plus d'informations sur le format Hayagriva, veuillez consulter sa documentation.

Pour insérer une bibliographie, il faut utiliser la commande `#bibliography()` en précisant le chemin du fichier de bibliographie.

Code

```
// Pour un fichier BibTeX
#bibliography("bibliographie/biblio.bib")

// Pour un fichier Hayagriva
#bibliography("bibliographie/biblio.yaml")
```

Pour plus d'informations sur la commande `#bibliography()`, veuillez consulter la documentation officielle de Typst.

Astuce

Zotero (avec le connecteur Better BibTeX) ou JabRef exportent directement votre bibliographie au format BibTeX. On peut noter que JabRef permet également d'exporter votre bibliographie au format Hayagriva.

6.1.2. Syntaxe de base

Les citations sont insérées dans le texte en utilisant soit la commande `#cite()`, soit en utilisant la même syntaxe que celle utilisée pour les références croisées `@cle`.

typ

```
La composition typographique scientifique
a été révolutionnée par Donald Knuth dans
les années 1970 @Knu84.
```

Rendu

La composition typographique scientifique a été révolutionnée par Donald Knuth dans les années 1970 [1].

6.1.3. Références multiples

Pour insérer plusieurs références dans le texte, il suffit de les chaîner.

typ

```
Typst est un nouveau langage de balise
développé par deux étudiants allemands,
Laurenz Mädje et Martin Haug, dans le
cadre de leur mémoire de M2 en
informatique @Mad22 @Hau22.
```

Rendu

Typst est un nouveau langage de balise développé par deux étudiants allemands, Laurenz Mädje et Martin Haug, dans le

cadre de leur mémoire de M2 en informatique [2, 3].

6.2. Glossaire, acronymes et nomenclatures

Plusieurs paquets Typst pour créer des glossaires et des acronymes existent. Le gabarit `community-cnam-thesis` n'intègre pas de paquets spécifiques pour créer des glossaires et des acronymes. Toutefois, il est possible d'utiliser les paquets suivants :

- **Glossaires**
  - Glossy
  - Glossarium
  - Gloss-awe
- **Acronymes**
  - Acrostiche
  - Acrotastic
  - Abbr
  - I-am-acro
- **Nomenclatures**
  - Nomos

Chapitre

7

Commentaires de relecture

Cette section présente les différentes fonctionnalités de relecture et d'annotation disponibles dans le template `community-cnam-thesis`. Ces fonctionnalités permettent aux auteurs et aux relecteurs de collaborer efficacement en ajoutant des commentaires, des notes de marge et des annotations directement dans le document.

Contenu

7.1. Commentaires collaboratifs 58
7.2. Types de commentaires 58
7.3. Syntaxe des annotations 59
7.4. Table des annotations 61

7.1. Commentaires collaboratifs

Pour activer les commentaires, il suffit d'insérer dans le fichier concerné la commande :

Code

```
#show: activate-comment
```

Pour désactiver les commentaires, il suffit d'insérer dans le fichier concerné la commande :

Code

```
#show: deactivate-comment
```

L'activation des commentaires créée une zone de marge sur le côté droit du document, où les commentaires peuvent être affichés.

Remarque

La commande `activate-comment` est un raccourci pour `marginalia.setup.with( ... )`, tandis que la commande `deactivate-comment` est un raccourci pour `page.with(margin: auto)`.

Si la commande d'activation est définie dans un fichier inclus, il n'est pas nécessaire de désactiver les commentaires dans les fichiers inclus suivants.

7.2. Types de commentaires

Le gabarit `community-cnam-thesis` propose différents types de commentaires, chacun ayant une icône spécifique pour les différencier visuellement :

- **`note`** : Utilisé pour ajouter des notes d'information ou des explications supplémentaires.
- **`comment`** : Utilisé pour ajouter des commentaires généraux ou des suggestions.
- **`question`** : Utilisé pour poser des questions ou demander des clarifications.
- **`todo`** : Utilisé pour indiquer des tâches à accomplir ou des éléments à vérifier.

7.3. Syntaxe des annotations

Remarque

L'environnement de commentaire a été activé pour cette section.

Les annotations peuvent être ajoutées en utilisant la commande `#comment()`, dont les arguments sont les suivants :

Paramètre

`(by): "Reviewer"` — `string` | `content`

Auteur du commentaire

Paramètre

`(type): "note"` — `string`

Type de commentaire

Paramètre

`(inline): false` — `bool`

Si `true`, le commentaire est inséré en ligne. Si `false`, il est inséré dans la marge.

Paramètre

`(color): blue` — `string` | `color`

Couleur de la boîte d'annotation

Paramètre

`(..args)` — `dictionary`

Arguments supplémentaires pour la personnalisation des annotations.

Remarque

La commande `#comment()` est construite à partir de la commande `#note()` fournie par le package `marginalia`, tandis qu'elle utilise la commande native `#block()` pour les notes en ligne. Par conséquent, `#comment()` hérite des paramètres de ces deux commandes. Pour plus d'informations sur les paramètres disponibles, veuillez consulter la documentation du package `marginalia`.

Attention

En raison de l'implémentation actuelle de la commande `#comment()`, certains paramètres de la commande `#inline-note()` ne sont pas encore pris en charge. C'est notamment le cas du paramètre `par-break`.

Ainsi, pour ajouter un commentaire de type `note` en marge dont la couleur de la boîte est bleue, il suffit d'utiliser la commande suivante :

Code

```
#comment(by: "Abbé Grégoire", color: blue)[Ceci est un commentaire
bleu.]
```

Comme vous pouvez le constater la commande précédente crée bien un commentaire de type `note` en marge dont la couleur de la boîte est bleue. [1]. Le lecteur attentif aura remarqué que l'annotation insérée est signalée dans le texte par son icône, sa couleur et son numéro.

1
Abbé Grégoire : Ceci est un commentaire bleu.

On peut aller plus loin en créant des boîtes d'annotation associées à un relecteur spécifique. Cela permet de différencier les commentaires de chaque relecteur par une couleur spécifique. Par exemple, créons deux relecteurs, l'Abbé Grégoire et Henri Tresca (qui sont les directeurs de cette thèse fictive).

Code

```
#let ab-comment = comment.with(by: "Abbé Grégoire", color: blue)
#let ht-comment = comment.with(by: "Henri Tresca", color: cnam-
color.primary)
```

En procédant ainsi, il est possible d'ajouter des commentaires de chaque relecteur en utilisant les commandes `ab-comment` et `ht-comment` [2] [3]. Ces notes ont été créées avec les commandes suivantes :

2
Henri Tresca : Je trouve cela génial !

3
Abbé Grégoire : Je ne suis pas sûr de comprendre cette remarque Henri ?

Code

```
En procédant ainsi, il est possible d'ajouter des commentaires de
chaque relecteur en utilisant les commandes `ab-comment` et `ht-
comment`#ht-comment(type: "comment", dy: -3.5em)[Je trouve cela
génial !]#ab-comment(type: "question")[Je ne suis pas sûr de
comprendre cette remarque Henri ?].
```

Les commentaires insérés dans les marges doivent généralement être relativement courts pour ne pas surcharger la mise en page. Cependant, il est possible d'ajouter des commentaires plus longs en utilisant l'argument `inline: true`. Cela permet d'insérer le commentaire directement dans le texte, ce qui est particulièrement utile pour les explications détaillées ou les discussions plus longues. [4]

4
Abbé Grégoire : Comme je n'ai pas d'idées pour faire un texte amusant et long, je place un classique `#lorem()`. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Il est également possible de surligner un passage du texte et d'y associer un commentaire. Pour ce faire, il suffit d'utiliser la commande `#highlight-comment()` qui permet de surligner le texte et d'ajouter un commentaire associé. La commande est la suivante :

Code

```
#highlight-comment(by: "Relecteur A", type: "comment", color: green,
highlight-body: [Texte surligné])[Commentaire associé]
```

Dans la section suivante, nous présenterons la création de la liste des annotations. [5]

[5] **Mathieu Aucejo :** Je me demande si cela est vraiment nécessaire ?

Remarque

La commande `#highlight-comment()` est construite à partir de la commande `#comment()`. Elle hérite des paramètres de cette dernière à l'exception du paramètre `inline`.

7.4. Table des annotations

Pour faciliter la lecture et la navigation dans les commentaires, le template `community-cnam-thesis` propose également une table des annotations. Cette table répertorie tous les commentaires ajoutés dans le document, avec leur numéro, leur auteur et leur type. Pour l'insérer dans le document, il suffit d'utiliser la commande `#listofnotes` à l'endroit souhaité.

Code

```
#listofnotes
```

Conclusion

Ce guide d'utilisation du template `community-cnam-thesis` pour Typst a été conçu pour aider les étudiants et les chercheurs à rédiger leur mémoire conformément aux normes du Conservatoire national des arts et métiers (Cnam). Il fournit des instructions détaillées sur la configuration du document, l'organisation des sections, la gestion des références bibliographiques, et l'utilisation des fonctionnalités avancées telles que les annotations.

Récapitulatif des fonctionnalités

Fonctionnalité	Syntaxe	Chapitre
Importation du gabarit	`#import "@local/community-cnam-thesis:0.1.4": *`	Chapitre 1
Initialisation du gabarit	`#show: community-cnam-thesis.with( ... )`	
Hiérarchisation des titres	`= Chapitre`, `== Section`, `=== Sous-section`	Chapitre 2
Chapitres non numérotés	`#show: chapter-nonum`	
Sections non numérotées	`== Section <nomnum-sec>`	
Références croisées	`<mon-label>` + `@mon-label`	
Environnements de mise en page	`front-matter`, `main-matter`, `appendix`	
Insertion de parties	`#part[Titre de la partie]`	
Quatrième de couverture	`backcover( ... )`	
Insertion d'une image	`image("chemin/vers/image.png")`	Chapitre 3
Insertion d'un tableau	`table( ... )`	
Figures/tableaux numérotés	`figure( ... )`	
Sous-figures	`subfigure( ... )`	
Équations en ligne	`$E=mc^2$`	Chapitre 4
Équations numérotées	`$ E=mc^2 $`	
Équations non numérotées	`$ E=mc^2 $ <nonum-eq>`	
Boîte d'information	`#info-box[ ... ]`	Chapitre 5
Boîte de conseil	`#tip-box[ ... ]`	
Boîte d'avertissement	`#warning-box[ ... ]`	
Boîte d'information importante	`#important-box[ ... ]`	
Boîte de démonstration	`#proof-box[ ... ]`	
Boîte de question	`#question-box[ ... ]`	
Boîte de code	`#code-box[ ... ]`	

Fonctionnalité	Syntaxe	Chapitre
Insertion d'une bibliographie	`#bibliography("chemin/vers/biblio.bib")`	
Citation dans le texte	`@cle`	Chapitre 6
Références multiples	`@cle1 @cle2`	
Activation des commentaires	`#show: activate-comment`	
Commentaire	`#comment( ... )[Texte du commentaire]`	Chapitre 7
Commentaire mis en évidence	`#highlight-comment( ... )[text][comment]`	

Bibliographie

[1] D. E. Knuth, *The TeXbook*, vol. A. dans Computers & Typesetting, vol. A. Reading, Massachusetts: Addison-Wesley, 1984.

[2] L. Mädje, « Typst: A programmable markup language for typesetting », Master's thesis, 2022.

[3] M. Haug, « Fast typsetting with incremental compilation », Master's thesis, 2022.

Annexes

ANNEXE

A

Validation PDF/A-1b

Cette annexe présente les instructions pour valider la conformité du document PDF généré avec le standard PDF/A-1b.

Contenu

A.1. Dépôt sur theses.fr 70
A.2. Validation CINES 70
A.3. Compilation Typst en PDF/A-1b 70

A.1. Dépôt sur theses.fr

La plateforme theses.fr permet de déposer les thèses et mémoires de recherche. Pour que le document soit accepté, il doit être conforme au standard PDF/A-1b (ISO 19005-1).

A.2. Validation CINES

Le service en ligne facile.cines.fr est proposé par le CINES (Centre Informatique National de l'Enseignement Supérieur) pour valider la conformité des documents PDF avec le standard PDF/A-1b. Il est recommandé d'utiliser ce service pour vérifier que votre document respecte les exigences de formatage avant de le soumettre à theses.fr.

A.3. Compilation Typst en PDF/A-1b

Typst prend en charge un certain nombre de standards de sortie PDF, y compris le format PDF/A-1b. Pour générer un document conforme à ce standard, plusieurs options sont possibles:

1. Ligne de commande

Code

```
typst compile main.typ thesis.pdf --pdf-standard a-1b
```

2. Si vous utilisez l'application web de Typst, vous pouvez sélectionner le format PDF/A-1b dans les options d'exportation avant de télécharger le document (voir Figure A.1).

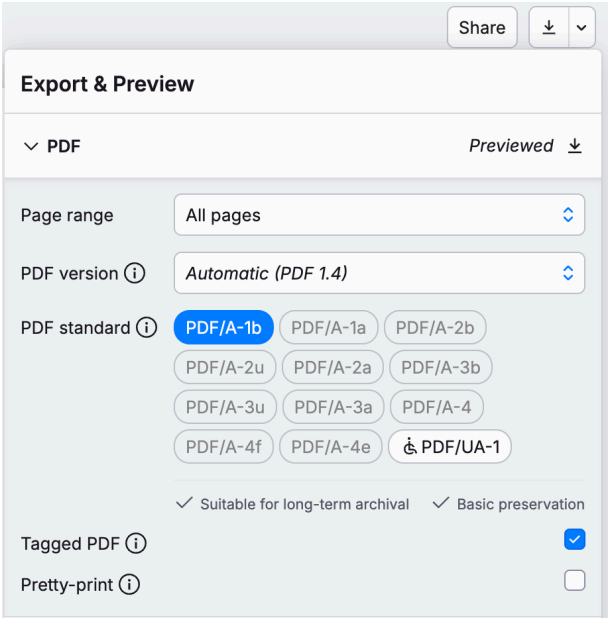


Figure A.1 – Sélection du format PDF/A-1b dans l'application web de Typst

3. Si vous utilisez l'extension Tinymist Visual Studio Code, vous pouvez configurer le format PDF/A-1b dans les paramètres de l'extension Typst. Pour ce faire, sélectionnez l'extension

→ Export Tool. Choisissez ensuite le format PDF/A-1b dans la liste déroulante « PDF Validator » (voir Figure A.2).

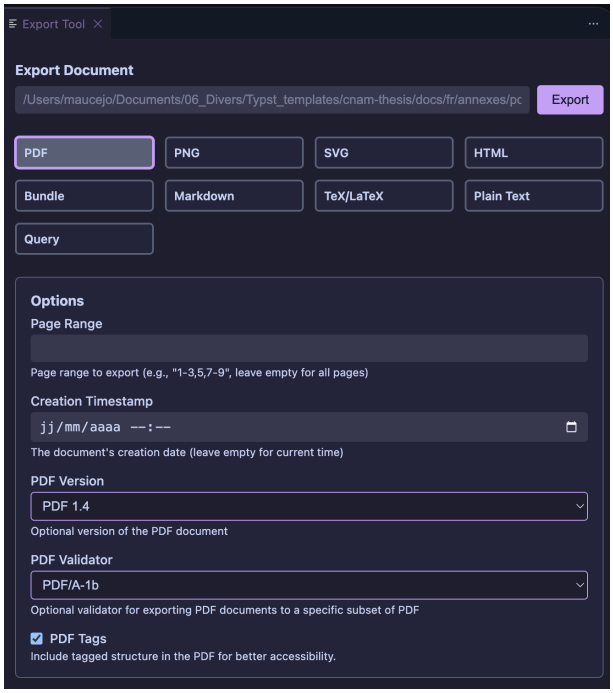


Figure A.2 – Sélection du format PDF/A-1b dans l'extension Tinymist de Visual Studio Code

⚠ Attention

Le format PDF/A-1b est un standard strict qui impose certaines contraintes sur le contenu et la structure du document. Par exemple, ce standard n'accepte pas les images avec de la transparence. Assurez-vous de vérifier votre document après la compilation pour vous assurer qu'il est conforme à ce standard.



le cnam

Mathieu Aucejo

Guide d'utilisation du gabarit Typst pour les thèses du Cnam

Résumé : Ce guide a pour objectif de présenter le template de thèse du Cnam, développé avec le langage de mise en page Typst. Il est destiné à aider les doctorants et les chercheurs à rédiger leur thèse conformément aux normes du Cnam, en utilisant un formatage cohérent et professionnel. Le guide couvre l'installation du template, la structure des fichiers, l'insertion de contenu, la gestion des références bibliographiques, et la personnalisation du document.

Abstract: This guide aims to present the Cnam thesis template, developed with the Typst typesetting language. It is intended to help doctoral students and researchers write their thesis in accordance with Cnam standards, using consistent and professional formatting. The guide covers template installation, file structure, content insertion, bibliographic reference management, and document customization.